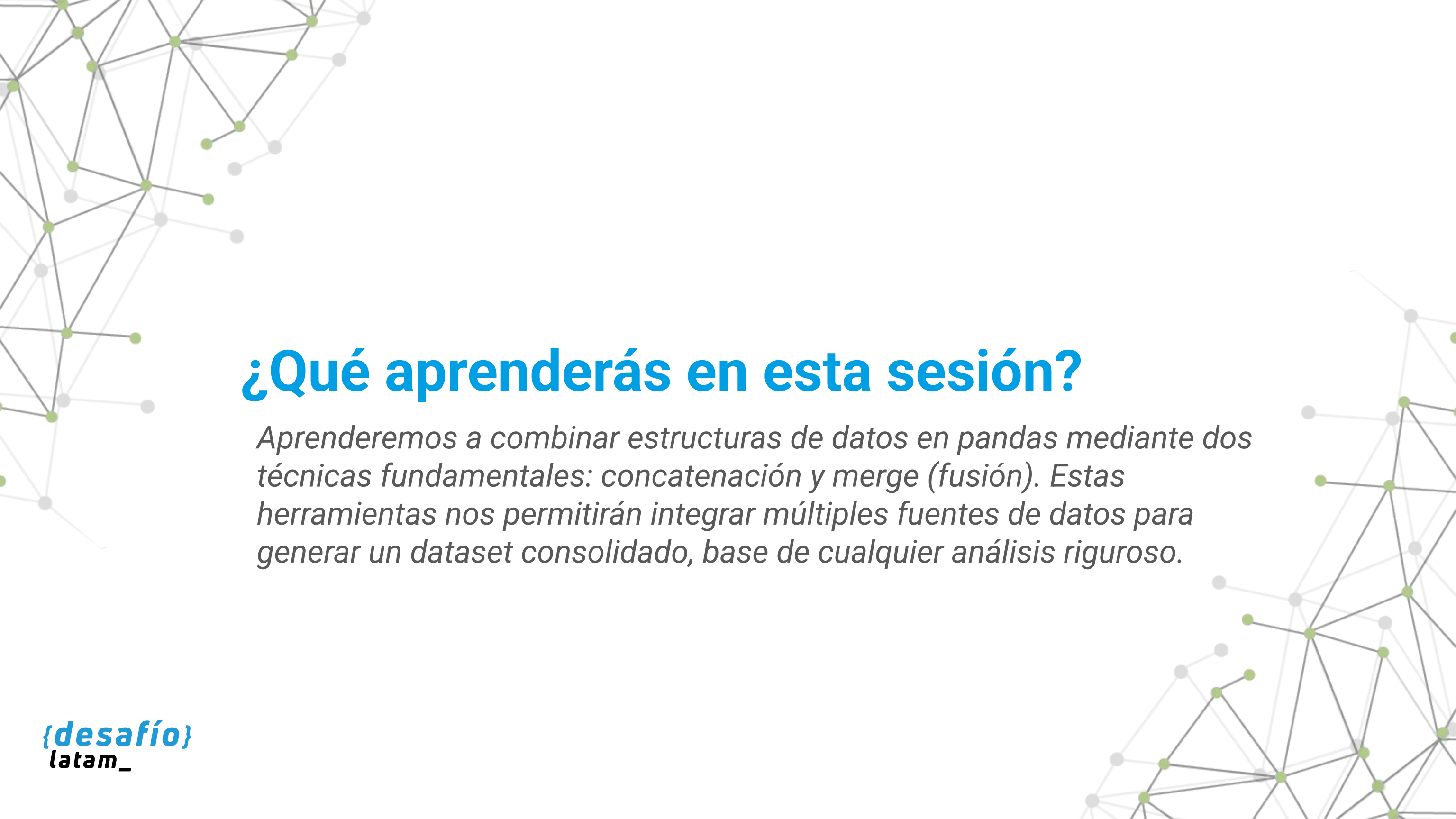




Limpieza y preparación de datos

Clase 7 - Combinación y fusión de dataframes





¿Qué aprenderás en esta sesión?

Aprenderemos a combinar estructuras de datos en pandas mediante dos técnicas fundamentales: concatenación y merge (fusión). Estas herramientas nos permitirán integrar múltiples fuentes de datos para generar un dataset consolidado, base de cualquier análisis riguroso.

Desafío inicial

Imagina que trabajas en una consultora de datos que colabora con una empresa del sector logístico. Recibes dos archivos distintos: uno contiene el detalle de los pedidos realizados, y otro, los datos de los clientes. Ambos deben combinarse para realizar un análisis de comportamiento por zona y frecuencia de compra.

¿Cómo puedes unir correctamente ambas fuentes de datos, considerando que no todos los clientes hicieron pedidos y que hay pedidos que podrían no estar correctamente vinculados?

Para resolverlo...deberás aprender a combinar conjuntos de datos utilizando concatenación y merge en pandas, identificando claves de combinación, comprendiendo tipos de join (left, right, inner, outer) y resolviendo posibles inconsistencias en los datos.

**/* Combinación y merge
de dataframes */**

Concatenación de dataframes

1

¿Qué es la concatenación?

La concatenación consiste en unir dataframes secuencialmente, ya sea apilando filas (uno sobre otro) o agregando columnas (uno al lado del otro). Es útil cuando tenemos datasets con la misma estructura y queremos integrarlos como si fueran parte de un único archivo.

2

Implementación

Esto se realiza mediante la función `pd.concat()` de pandas.



Ejemplo de concatenación

```
import pandas as pd
enero = pd.DataFrame({'producto': ['A', 'B'], 'ventas': [100, 150]})
febrero = pd.DataFrame({'producto': ['A', 'B'], 'ventas': [110, 160]})
```

En este ejemplo, tenemos dos dataframes con la misma estructura que representan ventas de productos en diferentes meses.

Resultado de la concatenación

```
ventas_total = pd.concat([enero, febrero], ignore_index=True)  
print(ventas_total)
```

Resultado:

producto ventas

0 A 100

1 B 150

2 A 110

3 B 160

tafframe Concatenation Result

Product	Month	Sales
A	January	100
B	January	150
A	February	110
B	February	160

Parámetros clave de concat

Parámetro	Descripción
axis=0	Une por filas (default)
axis=1	Une por columnas
ignore_index	Reinicia el índice
keys	Agrega etiquetas jerárquicas
join	Especifica si se conservan todas las columnas (outer) o solo las comunes (inner)

De estructuras homogéneas a heterogéneas

Hasta aquí hemos trabajado con estructuras homogéneas: mismos nombres de columnas, misma lógica.

Pero, ¿qué sucede si queremos cruzar información de distintas estructuras, como unir un pedido con los datos de quién lo hizo?

Para eso utilizamos una técnica más poderosa: merge.



/* Merge de dataframes */

Pandas Merge Operation



Merge de dataframes

¿Qué es merge y en qué se diferencia de la concatenación?

La función `merge()` en pandas permite unir dataframes en base a una o más columnas en común, tal como una operación JOIN en SQL. Esto es especialmente útil para vincular entidades relacionadas, como clientes y pedidos, productos y categorías, regiones y ventas, etc.

Sintaxis básica:

```
pd.merge(df1, df2, on='columna_clave', how='tipo_de_union')
```

Tipos de combinación (how)

'inner'

Solo registros que coincidan en ambas tablas

'left'

Todos los registros de la izquierda + coincidencias de la derecha

'right'

Todos los registros de la derecha + coincidencias de la izquierda

'outer'

Todos los registros, coincidan o no

tsheet



Table A	Key		Key	Table B
X	1	2	1	Y
x1		3	2	y1
x2			4	y2
x3				y3

Table A	Key		Key	Table B
X	1	2	1	Y
x1		3	2	y1
x2			4	y2
x3			NULL	y3

Table A	Key		Key	Table B
X	1	2	1	Y
x1		3	2	y1
x2		NULL	4	y2
x3				y3

Table A	Key		Key	Table B
X	1	2	1	Y
x1		3	2	y1
x2		NULL	4	y2
x3			NULL	y3

Ejemplo aplicado: unión de clientes y pedidos

```
clientes = pd.read_csv('clientes.csv')
pedidos = pd.read_csv('pedidos.csv')
df = pd.merge(pedidos, clientes,
on='id_cliente', how='left')
print(df)
```

Resultado esperado:

	id_pedido	id_cliente	monto	nombre
0	101	1	5000	Ana
1	102	2	7000	Luis
2	103	2	3000	Luis
3	104	5	2000	NaN

En este ejemplo, unimos la información de pedidos con los datos de clientes usando el id_cliente como clave.

Observa que el pedido 104 tiene un cliente (id=5) que no existe en la tabla de clientes, por lo que aparece como NaN.



Posibles errores comunes

- Usar `on='...'` sin que la columna exista en ambos dataframes.
- Tener nombres de clave distintos → solución: `left_on='...'`, `right_on='...'`
- Duplicados en las claves: produce combinaciones múltiples (comportamiento esperado).

Ahora que dominamos las dos formas principales de unir estructuras de datos en pandas, pasaremos a aplicarlas en un caso real. Usaremos archivos separados de clientes y pedidos para integrar la información comercial, resolver problemas de unión incompleta y generar un dataset útil para análisis posterior.

Actividad guiada: Combinación de clientes y pedidos



Ejercicio

Combinación de clientes y pedidos

Objetivo de la actividad

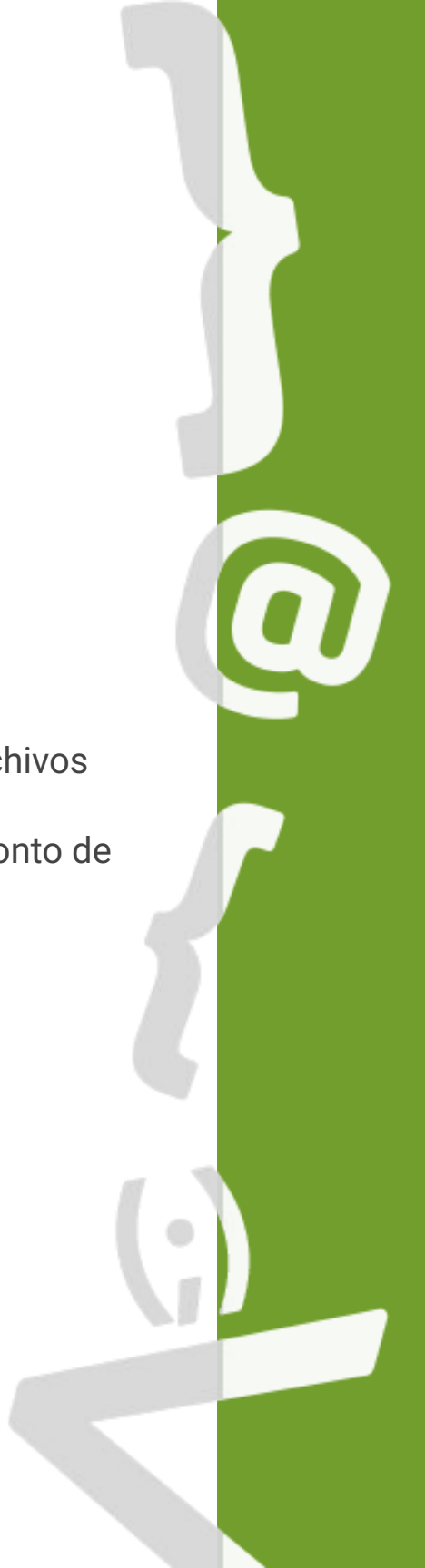
Aplicar técnicas de concat y merge para integrar dos fuentes de datos —clientes y pedidos— y construir un dataset consolidado para análisis comercial.

Situación problema

Trabajas en el área de análisis comercial de una empresa de retail que está preparando su informe trimestral de desempeño. Recibes dos archivos separados: uno contiene información básica de las personas clientas (nombre, ID), y el otro registra los pedidos realizados (ID de cliente y monto de compra).

Para responder a preguntas clave del equipo de ventas, necesitas consolidar esta información en un solo conjunto de datos que permita:

- Identificar qué personas realizaron pedidos y cuáles no.



Ejercicio

Instrucciones

Cargar los archivos

```
clientes = pd.read_csv('clientes.csv')
pedidos = pd.read_csv('pedidos.csv')
```

Concatenar nuevos registros

```
clientes_actualizados =
pd.concat([clientes, clientes],
ignore_index=True)
print(clientes_actualizados.head())
```

Unir con pedidos

```
df_final = pd.merge(pedidos, clientes,
on='id_cliente', how='left')
print(df_final)
```

Preguntas de reflexión y cierre

1

Pregunta 1

¿Qué sucede si un pedido tiene un id_cliente que no existe en la tabla de clientes?

2

Pregunta 2

¿Qué combinación (how) incluiría todos los pedidos, incluso sin cliente?

3

Pregunta 3

¿Cuál sería el efecto de tener claves duplicadas en ambos dataframes?



Actividad práctica autónoma: Unión de productos y proveedores



Ejercicio

Unión de productos y proveedores

Objetivo de la actividad

Aplicar merge() con distintos tipos de unión para analizar relaciones entre productos y proveedores.

Contexto

Has recibido dos archivos: productos.csv y proveedores.csv. Tu tarea es combinar ambos para:

- Identificar qué productos no tienen proveedor asignado.
- Verificar qué proveedores no tienen productos registrados.
- Preparar un informe para el área de abastecimiento.



Ejercicio

Instrucciones

1

Cargar los archivos

```
productos = pd.read_csv('productos.csv')  
proveedores = pd.read_csv('proveedores.csv')
```

2

Aplicar distintos merge()

```
# Unión izquierda: todos los productos  
pd.merge(productos, proveedores,  
on='id_proveedor', how='left')  
  
# Unión exterior: todo de ambos  
pd.merge(productos, proveedores,  
on='id_proveedor', how='outer')
```

3

Exportar resultados como CSV



Resumen

Comprendimos la diferencia entre concatenar y fusionar datasets.

Usamos `concat()` para apilar registros de estructura común.

Aplicamos `merge()` para combinar datos relacionales mediante claves.

Abordamos errores comunes y mejores prácticas en uniones.



Próxima sesión...

Prueba

{desafío}
latam_

